Universität Konstanz

Lerneinheit Informatik 11
Rechnerarchitektur II
Binärarithmetik und sequenzielle Logik

Kontaktstudium IMP

Universität Konstanz, 2019/2020

Inhalt

1. Binärarithmetik
    - Binärsystem als Schalter
    - Rechnen mit der ALU
        - Halbaddierer
        - Volladdierer
2. Sequenzielle Logik
    - bistabile Kippstufen (Flipflops)
    - SR-Riegel

1 Binärarithmetik

Inhalt

1. Binärsystem als Schalter
2. Rechnen mit der ALU
   - Halbaddierer
   - Volladdierer

Diese Folien sind eine überarbeitete Version der Folien des ersten IMP-Durchgangs von Herrn Christian Borgelt.

Binärsystem als Schalter

Die beiden Ziffern des Binärsystems 0 und 1 können direkt als Schalter implementiert werden.

Zifferndarstellung

Schalterzustand	Binärziffer
offen (Strom aus)	0
geschlossen (Strom an)	1

analog zur Logik

Schalterzustand	Wahrheitswert
offen (Strom aus)	0 (falsch)
geschlossen (Strom an)	1 (wahr)

Hinweis: Eine übliche Bezeichnung für eine einzelne Binärstelle ist das **Bit** (**bi**nary digi**t**).

einfache Berechnungen im Binärsystem:

- Test auf gerade / ungerade: Prüfen der geringwertigsten Stelle
- Multiplikation mit einer Zweierpotenz

(nicht in dieser Lerneinheit behandelt)

kompliziertere Berechnungen:

- Addition
- Subtraktion

Rechnen mit der ALU

Die ALU (arithmetic logical unit) ist das Herzstück eines Prozessors, in der arithmetische und logische Operationen auf gegebenen Operanden ausgeführt werden.

Die **Addition** ist die am häufigsten verwendete Operation.

Wiederholung (Additionsregeln der Bits a und b):

$$\begin{array}{rr} & 0 \\ + & 0 \\ \hline = & 0 \end{array} \qquad \begin{array}{rr} & 1 \\ + & 0 \\ \hline = & 1 \end{array} \qquad \begin{array}{rr} & 0 \\ + & 1 \\ \hline = & 1 \end{array} \qquad \begin{array}{rrr} & & 1 \\ + & 1 & 1 \\ \hline = & & 0 \end{array}$$

a	b	s	c
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

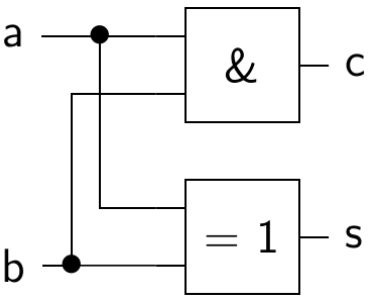
Ein Ein-Bit-Addierer benötigt somit **zwei** Ausgänge:

- Summe (s)
- Übertrag (c)

Halbaddierer (Ein-Bit-Addierer) in Logikgattern

Wir brauchen zwei unterschiedliche Logikgatter zur Darstellung der Ergebnisse. Ein Gatter (XOR) für die Summe (s) und ein Gatter (AND) für den Übertrag (c).

a	b	s	c
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



Hinweis: Das exklusive Oder (XOR) kann auch ausschließlich mit elementaren Gattern dargestellt werden. (siehe Zusatzmaterial)

Volladdierer in Logikgattern I

Wenn man nun mehr als ein Bit addieren möchte, so reicht ein einfacher Halbaddierer nicht mehr aus. Der Übertrag kann nämlich von einem Halbaddierer nicht beachtet werden.

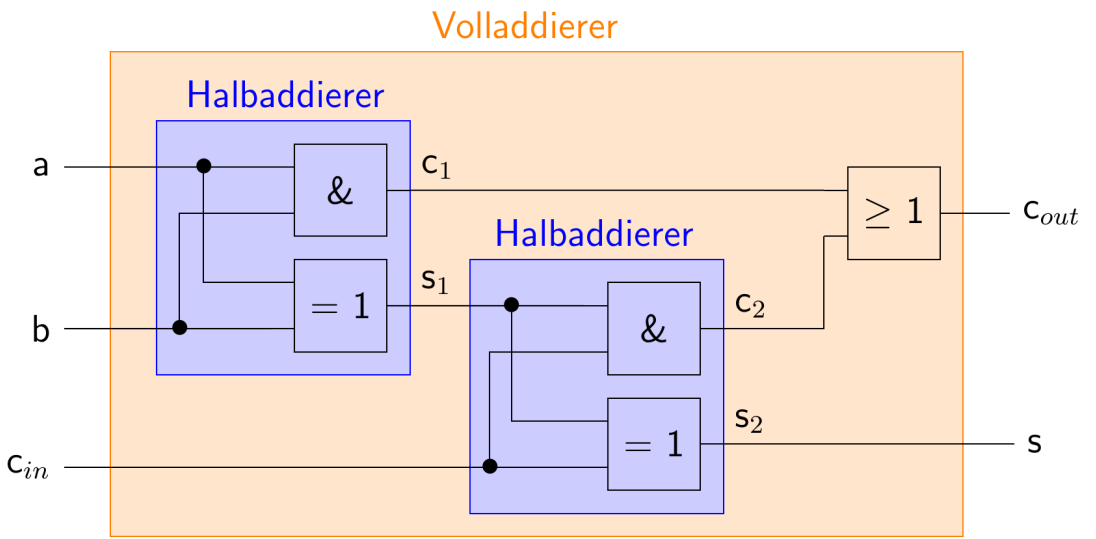
Der Volladdierer hat einen dritten Eingang, welcher den Übertrag aus der vorangehenden Addition beschreibt.

Beispiel (binäre 4-Bit-Addition):

a			1	0	1	1
b	+		0	1	1	1
c_{in}		1	1	1	1	
			0	0	1	0

Volladdierer in Logikgattern II

Ein Volladdierer berücksichtigt den Übertrag (c_{in}) einer vorangehenden Addition. Somit berechnet ein Volladdierer die Funktion $s = (a + b) + c_{in}$ und wird deswegen am einfachsten aus zwei Halbaddierern zusammengesetzt:



a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Volladdierer in Logikgattern II

Ein Volladdierer berücksichtigt den Übertrag (c_{in}) einer vorangehenden Addition. Somit berechnet ein Volladdierer die Funktion $s = (a + b) + c_{in}$ und wird deswegen am einfachsten aus zwei Halbaddierern zusammengesetzt:

Beispiel (binäre 3-Bit-Addition):

a		1	0	1
b	+	0	0	1
c_{in}		0	1	0
		1	1	0

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

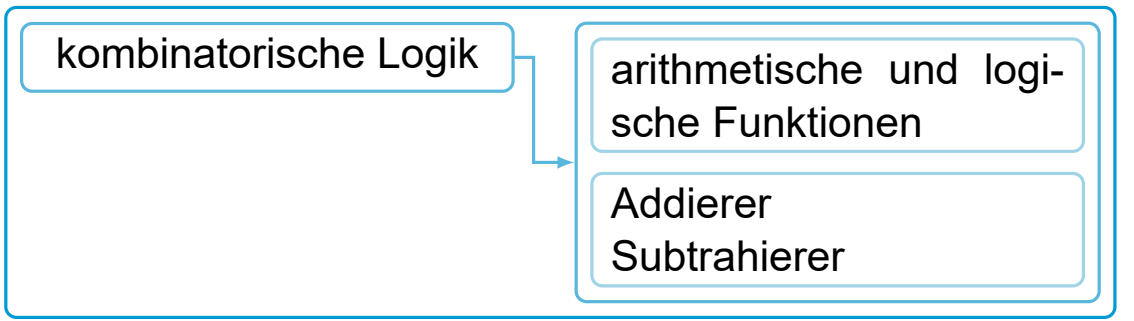
2 Sequenzielle Logik

Inhalt

1. Sequenzielle Logik
   - Flipflops
   - SR-Riegel

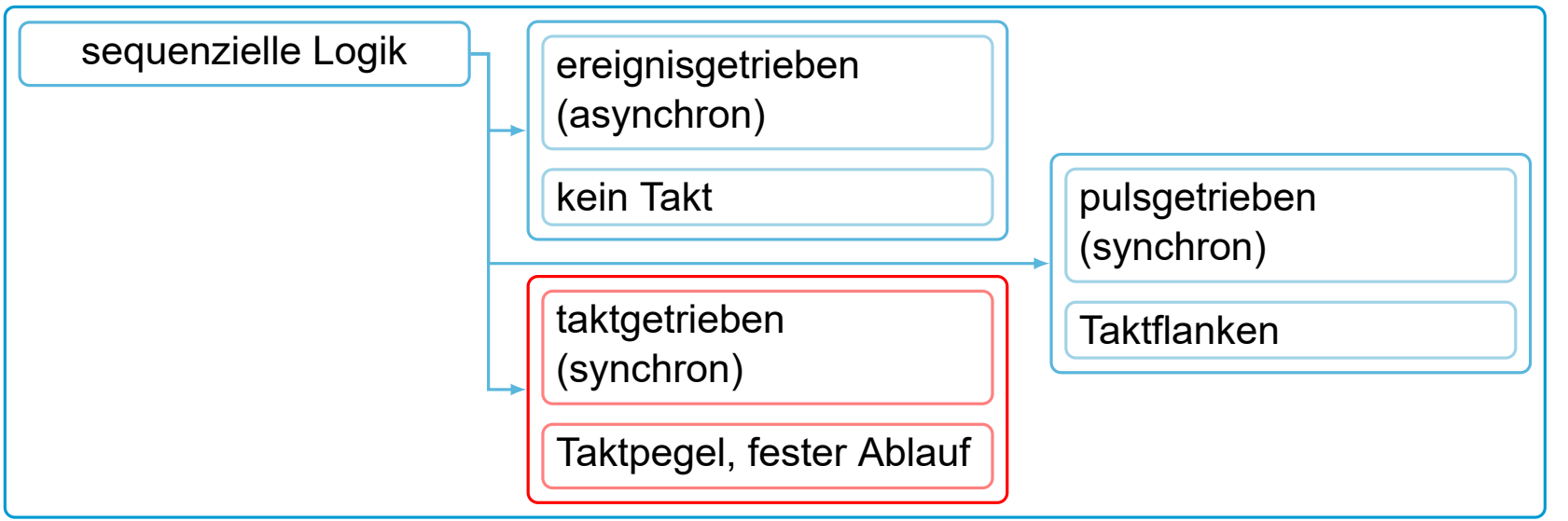
Vergleich zu kombinatorischer Logik

bisher - Schaltnetze



Vergleich zu kombinatorischer Logik

jetzt - Schaltwerke



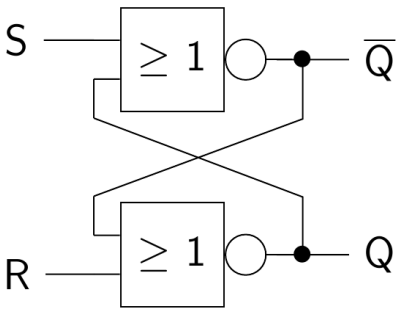
Rückkopplung und Takt

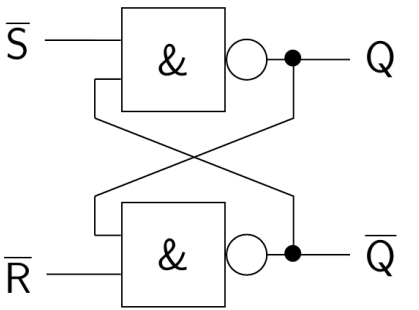
Informationen können mithilfe von **Rückkopplungen** in einem Schaltwerk gespeichert werden. Durch eine **Rückkopplung** werden Ausgänge wieder zu Eingängen einer Schaltung (siehe Flipflops).
Ein **Schaltnetz** (kombinatorische Logik) enthält keine Rückkopplungen und somit keine gerichteten Kreise.
Ein **Schaltwerk** (sequenzielle Logik) enthält Rückkopplungen und somit auch gerichtete Kreise.

Der **Takt** eines Schaltwerkes gibt an, in welchem zeitlichen Abstand die Änderungen in einem Schaltwerk passieren. Damit können die Zustandsveränderungen **synchron** ablaufen.

Flipflops

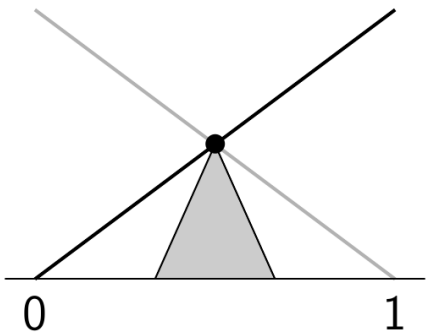
Eine **bistabile Kippstufe**, auch **bistabiles Kippglied** oder **Riegel** (*latch*), ist eine rückgekoppelte Schaltung aus zwei **NOR**- oder zwei **NAND**-Gattern mit zwei Eingängen und zwei Ausgängen (auch speziell **SR-Riegel** genannt).





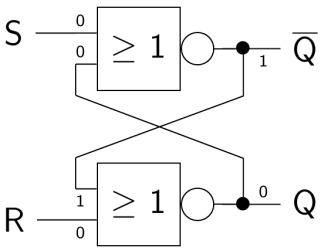
Flipflops

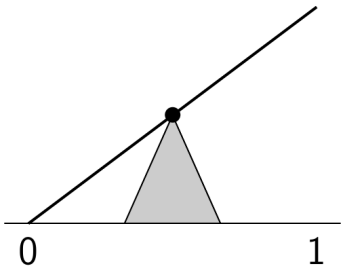
Oft wird einfach - aber ungenau - von **Flipflop** gesprochen.
Eine bistabile Kippstufe hat zwei stabile (daher auch *bistabil*) und einen metastabilen Zustand (mit inkonsistenten Ausgaben).
Man kann sich eine bistabile Kippstufe wie eine Wippe vorstellen, die ja ebenfalls in einer von zwei stabilen Lagen sein kann: der Balken ist links oder rechts unten.



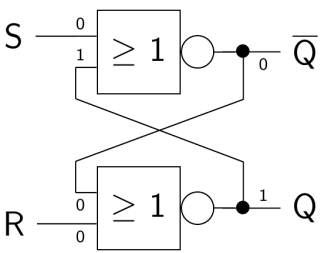
Einfache bistabile Kippstufen (SR-Riegel)

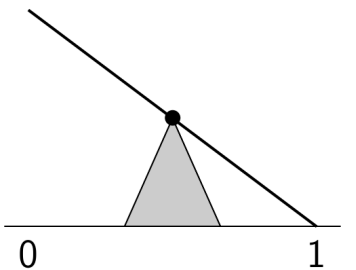
Wir betrachten die aus **NOR**-Gattern aufgebaute bistabile Kippstufe:





Die Ausgabe $Q = 0$ (und folglich $\overline{Q} = 1$) ist bei Eingaben $S = R = 0$ stabil.
$\Rightarrow$ einer von zwei stabilen Zuständen





Die Ausgabe $Q = 1$ (und folglich $\overline{Q} = 0$) ist bei Eingaben $S = R = 0$ stabil.
$\Rightarrow$ zweiter von zwei stabilen Zuständen

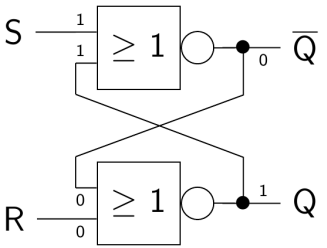
Einfache bistabile Kippstufen (SR-Riegel)

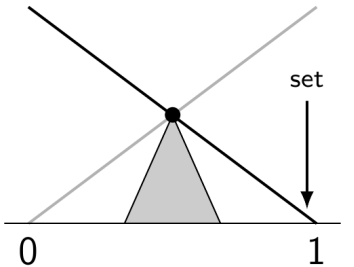
Die Eingaben $S = R = 0$ können also als Speicherzustand interpretiert werden, in dem die Kippstufe ihren Zustand (entweder 0 oder 1) beibehält.

Man beachte, dass die Ausgänge Q und $\overline{Q}$ stets komplementär zueinander sind.

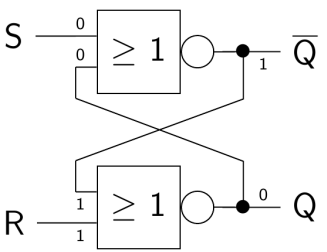
Die Eingaben $S = 1$ und $R = 0$ bzw. $S = 0$ und $R = 1$ können als Setzen (*set*) bzw. Rücksetzen (*reset*) des Zustandes interpretiert werden, durch welche die Kippstufe definiert in den Zustand 1 bzw. 0 gebracht wird.

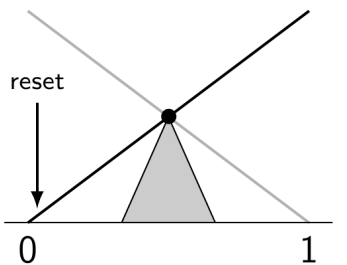
Einfache bistabile Kippstufen (SR-Riegel)





Die Ausgabe $Q = 1$ (und folglich $\overline{Q} = 0$) wird durch $S = 1$ und $R = 0$ gesetzt.
$\Rightarrow$ Zustand bleibt erhalten, wenn die Eingabe S von 1 auf 0 wechselt.

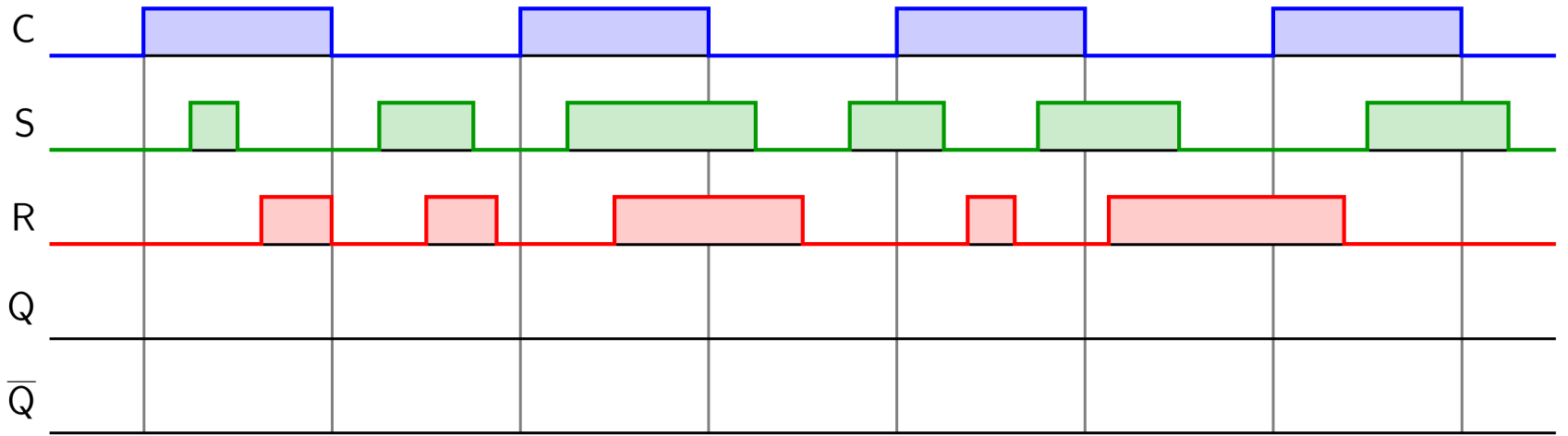




Die Ausgabe $Q = 0$ (und folglich $\overline{Q} = 1$) wird durch $S = 0$ und $R = 1$ gesetzt.
$\Rightarrow$ Zustand bleibt erhalten, wenn die Eingabe R von 1 auf 0 wechselt.

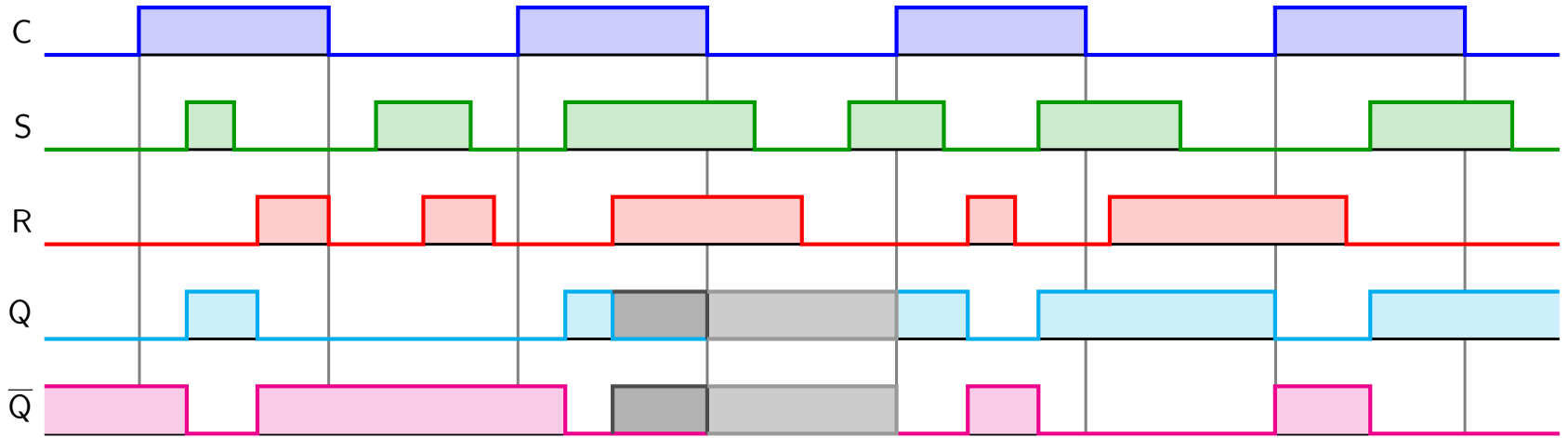
SR-Riegel und Takt

Die Änderung des Zustandes eines getakteten SR-Riegels kann man sehr schön anhand des folgenden Beispiels darstellen.



Hier sieht man den Takt (C), die Setzen- (S) und die Rücksetzen-Operationen (R).

SR-Riegel und Takt



Die Werte für Q und $\overline{Q}$ ergeben sich durch die Verbindung von S und R.
Ist der **Takt 0**, so bleibt der Zustand des SR-Riegels **gleich**.
Der **dunkelgraue** Bereich gibt an, dass der Zustand des SR-Riegels **inkonsistent** ist.
Der **hellgraue** Bereich gibt an, dass der Zustand des SR-Riegels **unbekannt** ist.